

# Graded Project - Parts 2 and 3

Technical Methodologies and Explainable AI

## E-commerce customer churn prediction: engineering practice and explainable AI

Implementation of the churn project framed in Part 1: Git and GitHub for versioning and CI, a cookiecutter project structure with one script per pipeline step, PEP8 tooling, Poetry for reproducibility, Sphinx documentation, and MLflow for experiment tracking, project packaging, model registry and serving. Part 3 adds explainability with SHAP: exact Shapley values for the champion model, global and per-class summaries, per-customer explanations and the reason codes the Part 1

### Code repository

[https://github.com/ribbzz/AI\\_Project\\_Methodology](https://github.com/ribbzz/AI_Project_Methodology)

### Group members

- Rabeeh Abou Ismail
- Amjad Bsar
- Majd Hammoud

|                    |                                                      |
|--------------------|------------------------------------------------------|
| <b>Dataset</b>     | E-Commerce Customer Churn (Kaggle) - 5,630 customers |
| <b>Deliverable</b> | Part 2 report - group dedicated repository           |
| <b>Date</b>        | September 2026                                       |

## 1. Summary

Part 1 of this project framed a customer churn prediction initiative for RetailGenius. Part 2 implements that framing as a working machine learning project on the E-Commerce Customer Churn dataset, with the emphasis the brief asks for: **the engineering practice around the model, not the model's score.**

### Code repository

[https://github.com/ribbzz/AI\\_Project\\_Methodology](https://github.com/ribbzz/AI_Project_Methodology) - the full project, including the dataset, the generated models and the MLflow tracking store, is committed there. Every command in this report can be run from a fresh clone.

## Requirements of the brief and where they are met

| Brief requirement                                                                  | Implementation                                                                                                                                             | In the repository                                        |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Git for code and model versioning, GitHub as the collaboration platform            | Ten focused commits; GitHub Actions runs formatting, linting, tests and the documentation build on every push; models are versioned in the MLflow registry | git history,<br>.github/workflows/ci.yml                 |
| Separate scripts for data preparation, feature engineering, training and inference | Four modules, each runnable on its own and exposed as a Make target, a Poetry script and an MLflow entry point                                             | src/churn/data, features, models                         |
| Cookiecutter project template                                                      | Cookiecutter Data Science layout, adapted to a src/ package                                                                                                | repository root                                          |
| Same Python version, PEP8 style                                                    | Python 3.11+ declared for the group; black, flake8 and pylint configured and passing (pylint 10.00/10)                                                     | pyproject.toml, setup.cfg,<br>.pylintrc                  |
| Dependency management                                                              | Poetry project, with pip requirement files and an MLflow python_env kept in step                                                                           | pyproject.toml,<br>requirements*.txt,<br>python_env.yaml |
| Automated documentation (Sphinx)                                                   | Sphinx with autodoc and napoleon, generated from Google-style docstrings                                                                                   | docs/source, make docs                                   |
| MLflow tracking - parameters, metrics, multiple runs in the UI                     | 9 training runs and 1 sweep parent run, each with parameters, 8 metrics, 4 plots and the model                                                             | mlflow.db, mlruns/                                       |
| MLflow Projects packaging                                                          | MLproject file with seven entry points                                                                                                                     | MLproject, python_env.yaml                               |
| MLflow Model Registry                                                              | Best run registered automatically as ecommerce-churn-classifier, alias champion                                                                            | train_model.register_best                                |
| Local serving with MLflow                                                          | mlflow models serve on the champion; a client script sends real customers to it                                                                            | make serve,<br>scripts/score_request.py                  |
| <b>Part 3:</b> SHAP TreeExplainer, per-point, per-class and global explanations    | src/churn/models/explain.py - waterfall, force, summary per class, mean SHAP, beeswarm and dependence plots, plus reason codes and a fairness audit        | make explain,<br>reports/figures/shap/                   |
| Deliverables: code, generated models, MLflow outputs                               | All committed; a helper rewrites the store's absolute paths after a clone                                                                                  | models/, mlruns/,<br>scripts/relocate_mlflow.py          |

## 2. Dataset and problem

The dataset is the Kaggle **E-Commerce Customer Churn Analysis and Prediction** workbook: one row per customer, 20 columns, 5,630 customers. The target, **Churn**, is 1 for 948 customers (**16.8%**). The raw file is committed in data/raw/ so that the pipeline runs without a Kaggle account.

| Group                 | Columns                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------|
| Identifier and target | CustomerId, Churn                                                                              |
| Relationship          | Tenure, DaySinceLastOrder, OrderCount, OrderAmountHikeFromlastYear, CouponUsed, CashbackAmount |
| Engagement            | HourSpendOnApp, NumberOfDeviceRegistered, PreferredLoginDevice                                 |
| Service experience    | SatisfactionScore, Complain, WarehouseToHome                                                   |
| Profile               | Gender, MaritalStatus, CityTier, NumberOfAddress, PreferredPaymentMode, PreferredOrderCat      |

## What the exploratory analysis changed in the pipeline

The notebook notebooks/1.0-eda.ipynb is committed with its outputs. Five findings shaped the code:

| Finding                      | Evidence                                                                                         | Consequence in the code                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Imbalanced target            | 16.8% churners; a model predicting 'no churn' for everyone is 83% accurate                       | Stratified split; class weights; PR-AUC and lift as the main metrics, accuracy not reported   |
| Missing values               | 7 numeric columns with 4.5-5.5% missing (DaySinceLastOrder the highest, 307 rows)                | Median imputation inside the sklearn pipeline, so training and serving treat gaps identically |
| Duplicate category spellings | Phone / Mobile Phone, CC / Credit Card, COD / Cash on Delivery, Mobile / Mobile Phone            | make_dataset.harmonise_categories merges them before encoding                                 |
| Strong early-life churn      | 65.5% of churners have a tenure of 0-1 month; median tenure is 1 for churners vs 10 for the rest | Engineered ratios normalised by tenure (orders, recency) and an IsNewCustomer flag            |
| Service friction matters     | Customers who complained churn at 31.7% vs 10.9%; single customers at 26.7%                      | Kept as raw features; they surface in the importance plot                                     |

## 3. Versioning and collaboration

The project lives in a GitHub repository shared by the group. The history is kept readable on purpose - each commit adds one coherent piece, so a reviewer can follow how the project was built:

| #  | Commit                                               | Content                                                 |
|----|------------------------------------------------------|---------------------------------------------------------|
| 1  | Set up project structure and tooling                 | layout, Poetry, requirements, linters, Makefile, config |
| 2  | Add raw dataset and data preparation step            | data/raw, make_dataset.py                               |
| 3  | Add feature engineering step                         | build_features.py, fitted preprocessor                  |
| 4  | Add model training with MLflow tracking and registry | train_model.py, evaluate.py, visualize.py               |
| 5  | Add batch inference and MLflow serving client        | predict_model.py, MLproject, serving scripts            |
| 6  | Add test suite and CI workflow                       | tests/, .github/workflows/ci.yml                        |
| 7  | Add Sphinx documentation                             | docs/source                                             |
| 8  | Add exploratory data analysis notebook               | notebooks/1.0-eda.ipynb                                 |
| 9  | Add MLflow runs, registered model and report figures | mlflow.db, mlruns/, reports/                            |
| 10 | Write README                                         | setup, usage, results                                   |

## What is versioned, and with which tool

| Artefact                                    | Tool                                            | Why                                                                                 |
|---------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------|
| Source code, configuration, notebooks, docs | Git / GitHub                                    | History, review and collaboration                                                   |
| Raw dataset (556 KB)                        | Git                                             | Small and immutable - the pipeline is reproducible from a clone                     |
| Intermediate and processed data             | Not versioned (git-ignored)                     | Fully regenerated by make data; committing them would duplicate the source of truth |
| Trained models                              | MLflow Model Registry, stored in the repository | Versions, aliases and lineage back to the run that produced each model              |
| Experiments                                 | MLflow tracking store (mlflow.db + mlruns/)     | Parameters, metrics and artefacts of every run, viewable in the UI                  |

## Continuous integration

GitHub Actions (.github/workflows/ci.yml) runs on every push and pull request, on Python 3.11 - the lower bound of the group standard. It fails the build if any of these fail:

```
black --check src tests scripts      # formatting
flake8 src tests                    # PEP8
pylint src/churn                    # static analysis
pytest --cov=churn                  # 21 tests
sphinx-build -W docs/source docs/build # docs, warnings are errors
```

**Recommended team workflow.** One feature branch per change, a pull request reviewed by another member, merge only on a green CI run, main protected. The CI makes 'it works on my machine' visible before it reaches the shared branch.

## 4. Code structure

The layout follows **Cookiecutter Data Science**, adapted in two ways: the code is an installable package under src/ (so tests and scripts import it the same way), and MLflow's store sits at the root where every command can find it.

```

AI_Project_Methodology/
├─ data/
│  └─ raw/                E_Commerce_Dataset.xlsx (immutable)
│  └─ interim/            clean.parquet (generated)
│  └─ processed/          train / test.parquet (generated)
├─ docs/source/           Sphinx: index, usage, API reference
├─ models/                preprocessor.joblib
├─ mlflow.db, mlruns/     tracking store, artefacts, model registry
├─ notebooks/1.0-eda.ipynb exploratory analysis
├─ reports/               figures/, model_comparison.csv, predictions.csv
├─ scripts/               score_request.py, report_figures.py, relocate_mlflow.py
├─ src/churn/
│  └─ config.py           every path and constant, defined once
│  └─ data/make_dataset.py STEP 1 data preparation
│  └─ features/build_features.py STEP 2 feature engineering
│  └─ models/train_model.py STEP 3 training, tracking, registry
│  └─ models/predict_model.py STEP 4 batch inference
│  └─ models/evaluate.py  metrics
│  └─ visualization/visualize.py evaluation plots
├─ tests/                 test_pipeline.py, test_training.py
├─ MLproject, python_env.yaml MLflow project
├─ pyproject.toml         Poetry + black/pytest configuration
├─ requirements.txt, requirements-dev.txt
├─ setup.cfg, .pylintrc   flake8 / pylint configuration
└─ Makefile               one target per step

```

## The four pipeline steps

*data/raw -> data/processed*

*reports/predictions.csv*

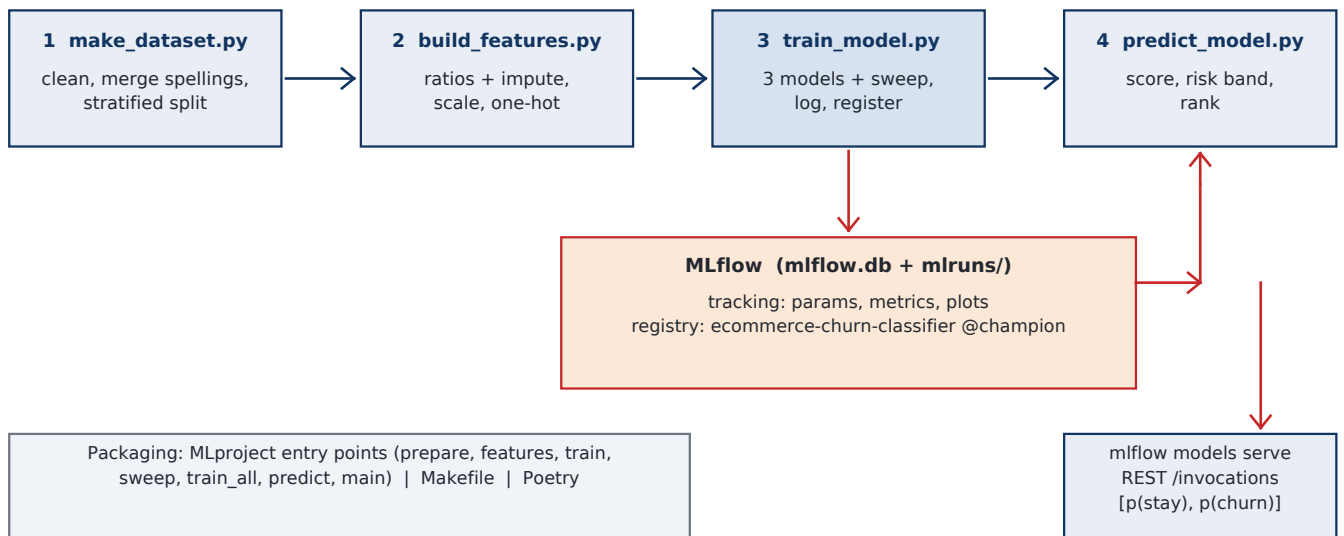


Figure 1 - Pipeline steps, the MLflow store they write to, and the two ways a model leaves it: batch scoring and the REST endpoint.

| Step                  | Module                     | Input -> output                                   | Key design choice                                                                                                           |
|-----------------------|----------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1 Data preparation    | data/make_dataset.py       | raw workbook -> clean.parquet, train/test.parquet | Only cleaning, no modelling decisions - the rules can be reviewed by a business reader                                      |
| 2 Feature engineering | features/build_features.py | train.parquet -> preprocessor.joblib              | One function, feature_frame(), is used by training, batch inference and the serving client, so the three cannot drift apart |

| Step        | Module                  | Input -> output                             | Key design choice                                                                |
|-------------|-------------------------|---------------------------------------------|----------------------------------------------------------------------------------|
| 3 Training  | models/train_model.py   | train/test -> MLflow runs, registered model | Preprocessing and estimator are one sklearn Pipeline, logged as a single model   |
| 4 Inference | models/predict_model.py | any cleaned table -> predictions.csv        | Loads the model by registry alias, not by file path; adds a risk band and a rank |

Each step is a module with a `main()` function and an `argparse` interface, so the same code is reachable four ways: `python -m churn.<step>`, a Make target, a Poetry script (`churn-data`, `churn-features`, `churn-train`, `churn-predict`) and an MLflow entry point. **config.py** is the only place where paths, column lists and constants are written - no filename appears twice in the project.

## 5. Standardization

### Python version

The group standard is **Python 3.11 or newer**, declared in `pyproject.toml` (`python = ">=3.11,<3.15"`). CI enforces the lower bound by running on 3.11; the project was developed and tested locally on 3.14. Version ranges were chosen instead of exact caret pins after a real problem: a pin of `pyarrow ^17` forced pip to build `pyarrow` from source on Python 3.14, where no pre-built package of that version exists (see section 12).

### PEP8 tooling

| Tool   | Role                                            | Configuration                                                                      | Result             |
|--------|-------------------------------------------------|------------------------------------------------------------------------------------|--------------------|
| black  | Formatter - removes style debates               | line length 100 ( <code>pyproject.toml</code> )                                    | 16 files unchanged |
| flake8 | PEP8 checker                                    | max line 100, E203/W503 ignored for black compatibility ( <code>setup.cfg</code> ) | 0 warnings         |
| pylint | Static analysis: naming, docstrings, complexity | <code>.pylintrc</code>                                                             | 10.00 / 10         |

The one pylint finding during development was instructive: `train_one()` took six positional arguments. Rather than silencing the rule, the optional ones (`run_name`, `nested`) were made keyword-only, which also makes every call site self-describing.

### Docstrings

Every public module and function has a Google-style docstring (Args / Returns). They are written for the reader who needs the **why** - for example, why the split is stratified or why the served model returns probabilities - and they are what Sphinx renders into the API reference.

## 6. Reproducibility

| Layer                 | Mechanism                                                                                                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dependencies          | <b>Poetry</b> (pyproject.toml) is the reference. requirements.txt and requirements-dev.txt mirror it for plain pip and for CI; python_env.yaml points MLflow at the same list |
| Environment isolation | A virtual environment per clone (.venv, git-ignored); MLflow projects run with --env-manager local or build their own virtualenv from python_env.yaml                         |
| Randomness            | A single RANDOM_STATE = 42 in config.py feeds the split and every estimator                                                                                                   |
| Data                  | Raw file committed; every downstream file is regenerated by make data / make features                                                                                         |
| Model inputs          | The MLflow model signature records the 23 input columns and their types; numeric inputs are cast to float so a missing value never breaks schema enforcement                  |
| Execution             | make all rebuilds everything from the raw file; mlflow run . -e main does the same through MLflow                                                                             |

### Reproducibility, observed

The default LightGBM run and the sweep run with learning\_rate=0.05, num\_leaves=31 use identical settings. They were trained separately and produced identical metrics to the last digit (PR-AUC 0.99901, Brier 0.00449) - the fixed seed and the deterministic pipeline doing their job.

### Getting started from a clone

```
git clone https://github.com/ribbzz/AI_Project_Methodology.git
cd AI_Project_Methodology
poetry install && poetry shell      # or: python -m venv .venv && make install
make all                          # data -> features -> train + sweep + register -> predict
make ui                            # MLflow UI on http://127.0.0.1:5000
```

## 7. Documentation

Documentation is generated with **Sphinx** from the code itself, so it cannot fall out of date with the docstrings. make docs builds it into docs/build/ (git-ignored); CI builds it with warnings treated as errors.

| Component     | Choice                                                                                                                                                                |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Extensions    | autodoc (API from docstrings), napoleon (Google style), autosummary, viewcode (links to highlighted source), intersphinx (links to Python, pandas, scikit-learn docs) |
| Theme         | sphinx_rtd_theme                                                                                                                                                      |
| Pages         | index - project overview and the four steps; usage - installation, pipeline, MLflow project, serving, quality gates; api - every module of the package                |
| Heavy imports | mlflow and lightgbm are mocked (autodoc_mock_imports), so the docs build fast and without the ML stack                                                                |

## 8. Model tracking and deployment with MLflow

MLflow 3 is used for all four capabilities named in the brief. The tracking store is a SQLite database (mlflow.db) plus an artefact directory (mlruns/), both pinned to the project root in config.py so that every script writes to the same place whatever directory it is launched from.

### 8.1 Tracking - parameters, metrics and multiple runs

Every call to `train_one()` opens a run and records:

| Logged item | Content                                                                                                                |
|-------------|------------------------------------------------------------------------------------------------------------------------|
| Parameters  | model family, decision threshold, train/test sizes, and every scalar hyper-parameter of the estimator (prefixed model) |
| Metrics     | pr_auc, roc_auc, lift_at_10pct, recall_at_10pct, precision, recall, f1, brier                                          |
| Tags        | dataset file, model family; the sweep parent also records its grid and best child                                      |
| Artefacts   | plots/confusion.png, curves.png (ROC + precision-recall), lift.png, importance.png                                     |
| Model       | the whole preprocessing + estimator pipeline, with signature and input example                                         |

The experiment **ecommerce-churn** holds ten runs: three model families (logistic regression, random forest, LightGBM) and a LightGBM sweep over `learning_rate` x `num_leaves`, logged as six **nested runs** under one parent so that the UI groups them.

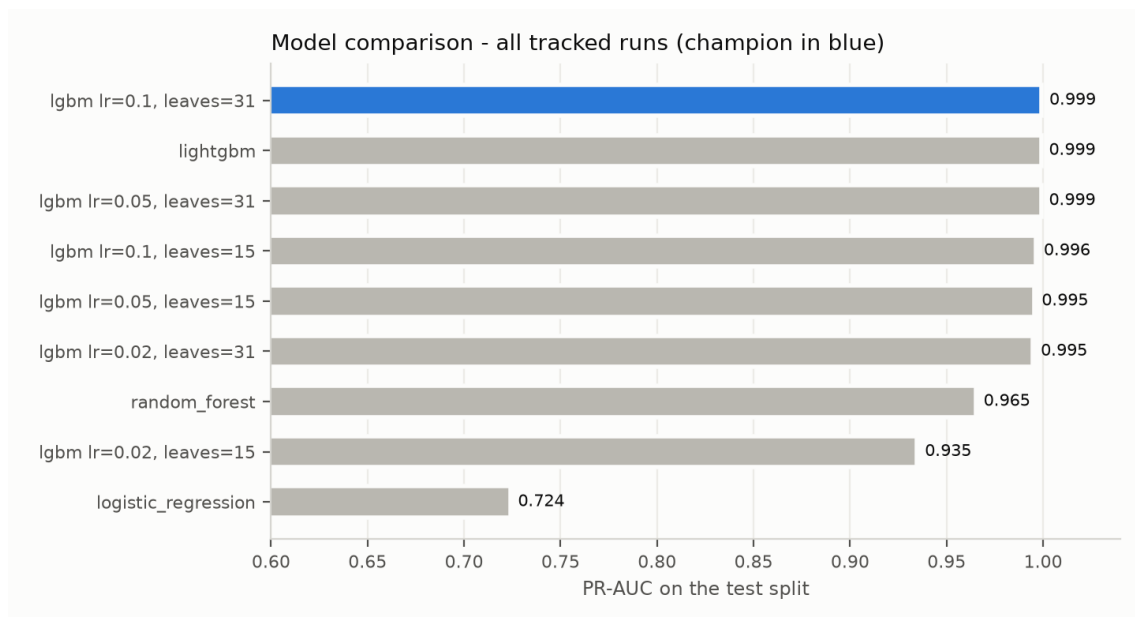


Figure 2 - PR-AUC of every tracked run, generated from the tracking store by `scripts/report_figures.py`.

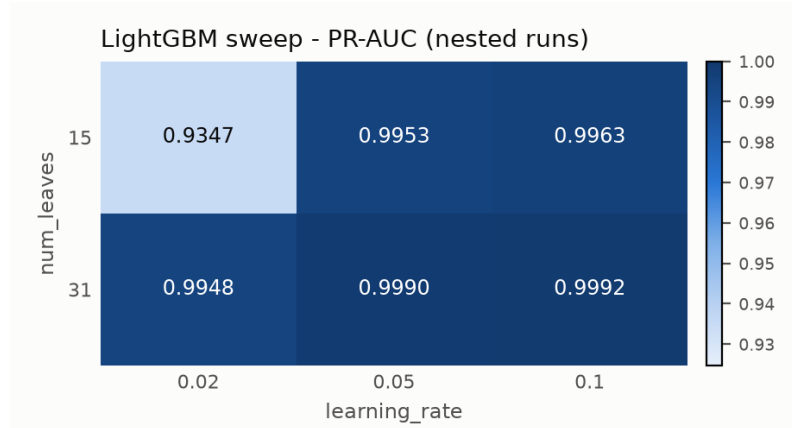
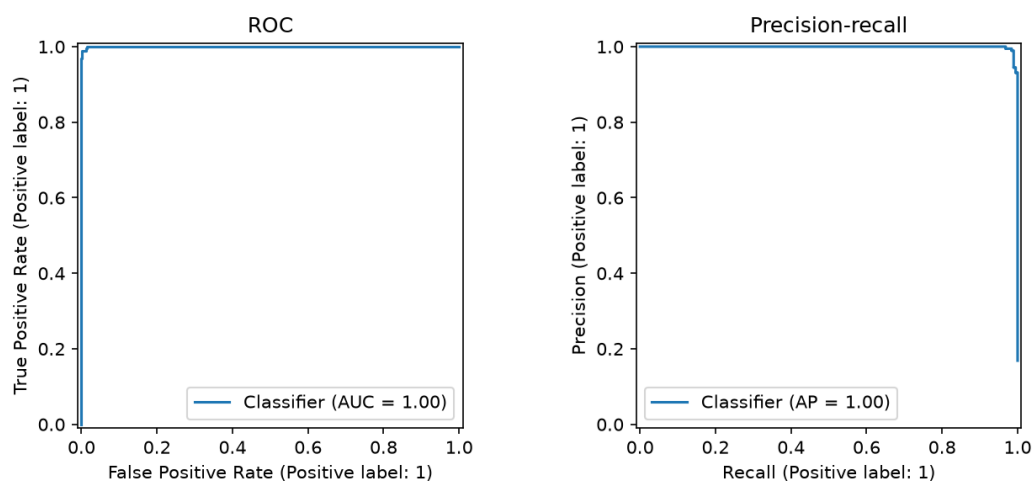


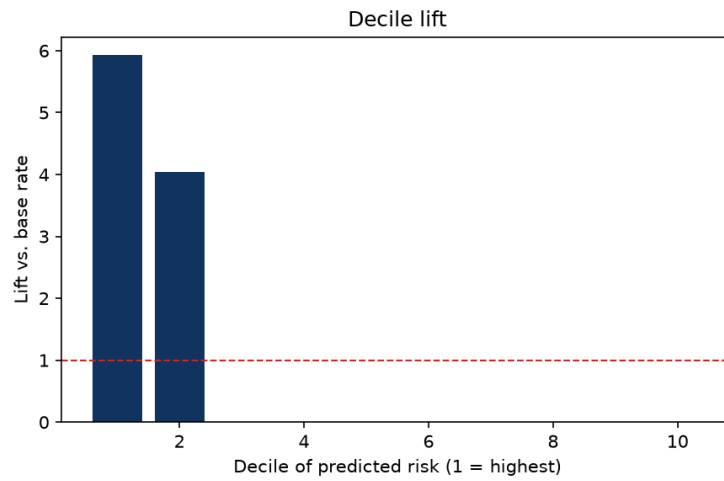
Figure 3 - The nested LightGBM sweep. Small trees with a slow learning rate under-fit within 600 iterations; the other settings are within 0.005 of each other.

| Run                               | PR-AUC        | ROC-AUC | Lift @10% | Recall @10% | Precision | Recall | Brier  |
|-----------------------------------|---------------|---------|-----------|-------------|-----------|--------|--------|
| <b>LightGBM lr=0.1, leaves=31</b> | <b>0.9992</b> | 0.9998  | 5.93      | 0.59        | 0.99      | 0.99   | 0.0035 |
| LightGBM (defaults)               | 0.9990        | 0.9998  | 5.93      | 0.59        | 0.98      | 0.98   | 0.0045 |
| LightGBM lr=0.1, leaves=15        | 0.9963        | 0.9993  | 5.93      | 0.59        | 0.97      | 0.99   | 0.0052 |
| LightGBM lr=0.05, leaves=15       | 0.9953        | 0.9991  | 5.93      | 0.59        | 0.91      | 1.00   | 0.0152 |
| LightGBM lr=0.02, leaves=31       | 0.9948        | 0.9990  | 5.93      | 0.59        | 0.91      | 0.99   | 0.0173 |
| Random forest                     | 0.9651        | 0.9928  | 5.87      | 0.58        | 0.88      | 0.87   | 0.0372 |
| LightGBM lr=0.02, leaves=15       | 0.9347        | 0.9884  | 5.56      | 0.55        | 0.79      | 0.96   | 0.0435 |
| Logistic regression               | 0.7239        | 0.9035  | 4.55      | 0.45        | 0.52      | 0.84   | 0.1181 |

Test split, 1,126 customers, threshold 0.5. The LightGBM lr=0.05, leaves=31 sweep run is identical to the defaults run and is omitted.

### Evaluation plots of the champion run (MLflow artefacts)





Figures 4-5 - ROC and precision-recall curves; decile lift. The first decile reaches the maximum possible lift on this split.

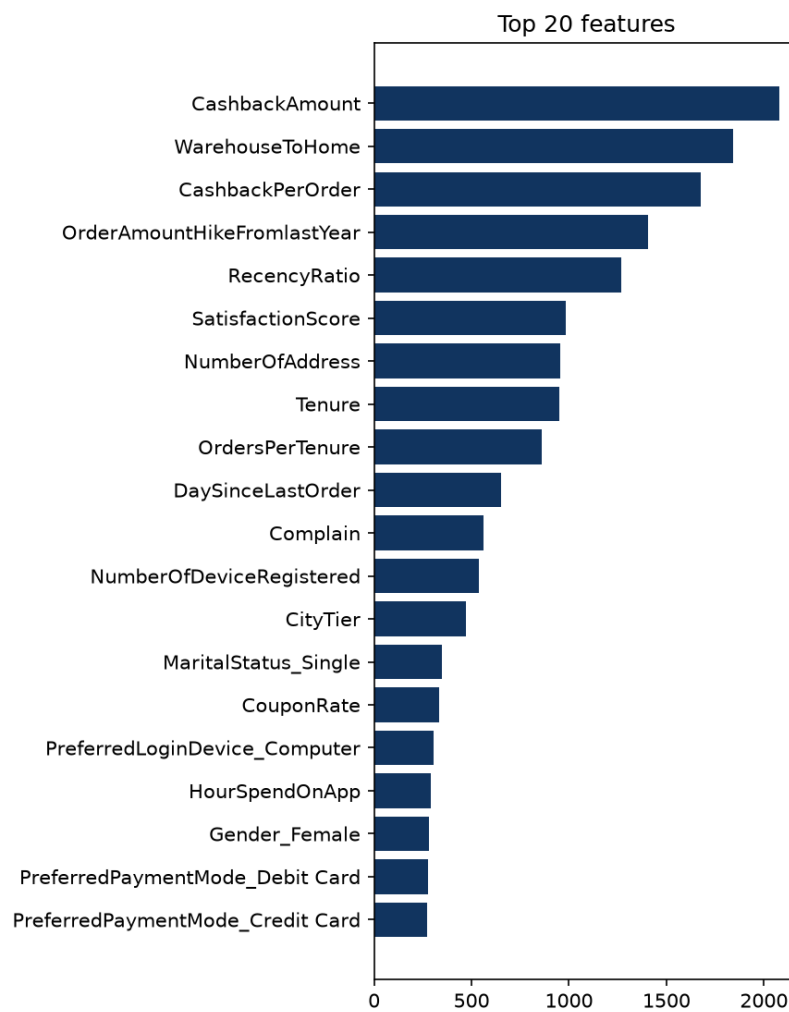


Figure 6 - Feature importance of the champion (LightGBM split counts). Tenure, cashback and the engineered ratios lead.

## 8.2 MLflow Projects - packaging

The repository is an MLflow project. The MLproject file names the environment (python\_env.yaml) and seven entry points, so the whole pipeline can be run by someone who only has MLflow installed:

| Entry point | Parameters                        | Command                                             |
|-------------|-----------------------------------|-----------------------------------------------------|
| prepare     | -                                 | python -m churn.data.make_dataset                   |
| features    | -                                 | python -m churn.features.build_features             |
| train       | model (lightgbm), threshold (0.5) | train_model --model {model} --threshold {threshold} |
| sweep       | -                                 | train_model --sweep                                 |
| train_all   | -                                 | train_model --all --sweep --register                |
| predict     | input_path, alias (champion)      | predict_model --input {input_path} --alias {alias}  |
| main        | -                                 | prepare, features and train_all chained             |

```
mlflow run . -e main --env-manager local
mlflow run . -e train -P model=random_forest -P threshold=0.4 --env-manager local
```

## 8.3 Model Registry - versioning and management

After training, register\_best() picks the run with the highest PR-AUC and registers its model as **ecommerce-churn-classifier**. The version is given the alias **champion**, two tags (selected\_on = pr\_auc, run\_name) and a description with its scores.

| Registry element | Value                                                                                     |
|------------------|-------------------------------------------------------------------------------------------|
| Registered model | ecommerce-churn-classifier                                                                |
| Version 1        | LightGBM, learning_rate=0.1, num_leaves=31 - PR-AUC 0.9992, lift@10% 5.93                 |
| Alias            | champion -> version 1                                                                     |
| Lineage          | the version links back to its run: parameters, metrics, plots and code are one click away |

**Aliases rather than stages.** MLflow 3 deprecates the fixed None / Staging / Production / Archived stages in favour of aliases. Consumers load models:ecommerce-churn-classifier@champion; promoting a new model, or rolling back, is a single alias move with no code change and no redeployment - the champion / challenger practice from Part 1 made concrete.

## 8.4 Serving - local inference server

The champion is served over REST by MLflow's inference server (FastAPI + uvicorn). make serve first points the store at the current checkout, then starts it:

```
MLFLOW_TRACKING_URI=sqlite:///mlflow.db \
  mlflow models serve -m "models:/ecommerce-churn-classifier@champion" -p 5001 --env-manager local

python scripts/score_request.py --port 5001      # POST /invocations
```

The model is logged with **pyfunc\_predict\_fn="predict\_proba"**, so the endpoint returns [p(stay), p(churn)] for each customer instead of a hard 0/1. A label cannot be ranked, and ranking is exactly what the retention team needs (Part 1, section 4.3). The client sends three churners and three non-churners taken from the test split:

| CustomerID | actual_churn | churn_probability |
|------------|--------------|-------------------|
| 52126      | 1            | 0.9999            |
| 50959      | 1            | 0.9999            |
| 55217      | 1            | 0.9998            |
| 54007      | 0            | 0.0000            |
| 51970      | 0            | 0.0000            |
| 54236      | 0            | 0.0000            |

## 8.5 Batch inference

make predict loads the champion by alias, scores the test split and writes reports/predictions.csv with the probability, a **risk band** and the rank. Bands use the thresholds in config.py (HIGH  $\geq 0.60$ , MEDIUM  $\geq 0.35$ ):

| Band   | Customers | Actual churn rate |
|--------|-----------|-------------------|
| HIGH   | 190       | 98.9%             |
| MEDIUM | 2         | 0.0%              |
| LOW    | 934       | 0.2%              |

## 9. Explainable AI with SHAP (part 3)

Part 1 promised marketing a risk score **and the top three reasons behind it**; part 2 delivered the score. Part 3 delivers the reasons, using **SHAP** (SHapley Additive exPlanations). The champion is a LightGBM model, so Shapley values are computed **exactly** with shap.TreeExplainer rather than approximated by sampling.

### 9.1 What the brief asks, and where each item is

| Required                                              | Implementation                                                         | Output                                                 |
|-------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------|
| SHAP installed and declared                           | shap $\geq 0.46$ added to requirements.txt and pyproject.toml          | make explain                                           |
| TreeExplainer and Shapley values                      | shapley_values() in src/churn/models/explain.py                        | 1,126 x 35 value matrix                                |
| Explain one specific point                            | Waterfall and force plot for one customer                              | waterfall_customer_54023.png, force_customer_54023.png |
| Explain all points at once                            | Interactive stacked force plot                                         | force_all_customers.html                               |
| Summary plot per class                                | Summary for 'churn' and its mirror image for 'no churn'                | summary_class_churn.png, summary_class_no_churn.png    |
| Waterfall / force / mean SHAP / beeswarm / dependence | All five produced by make explain                                      | 11 files in reports/figures/shap/                      |
| Going further                                         | Per-customer reason codes and a fairness audit of sensitive attributes | predictions_with_reasons.csv, fairness_audit.csv       |

## 9.2 How it is wired into the project

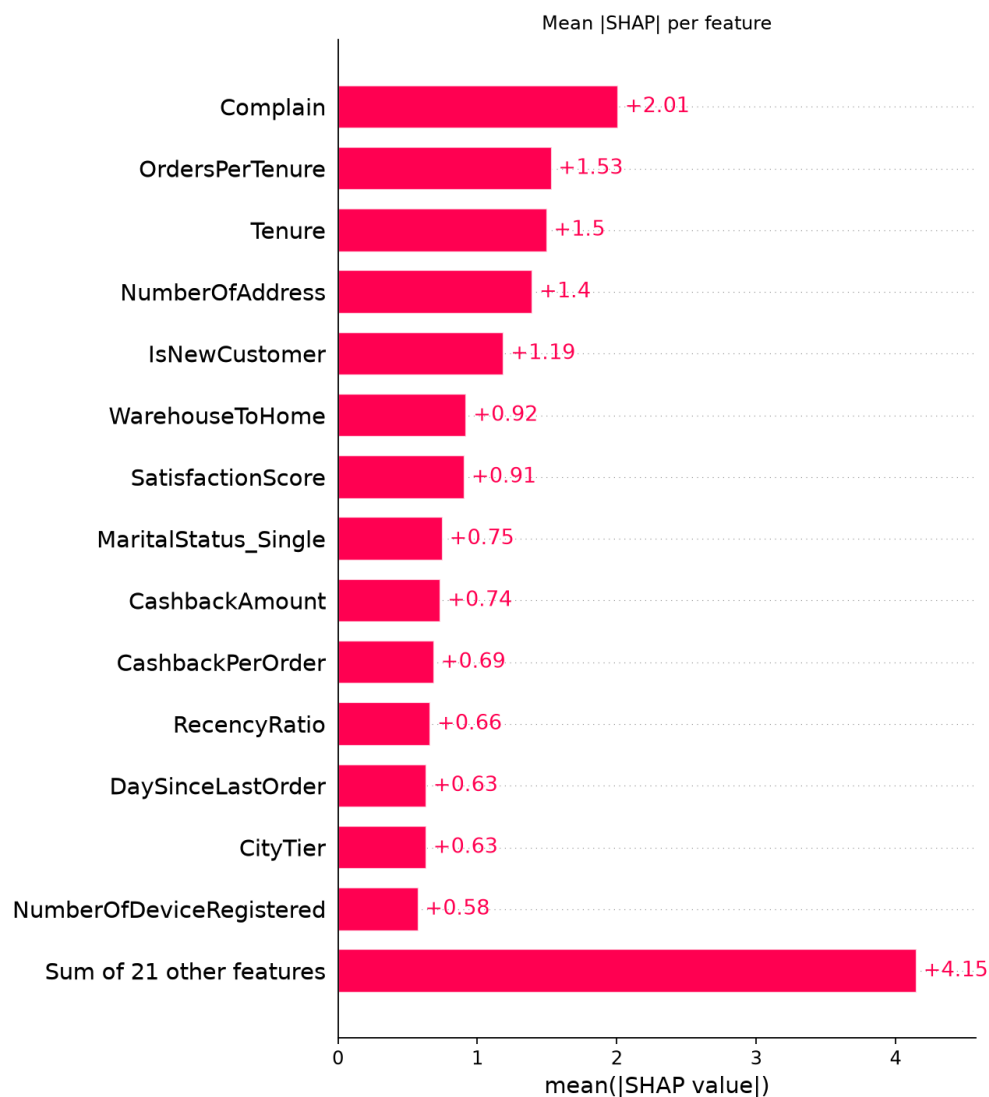
The registered champion is a **pipeline**: preprocessing then estimator. TreeExplainer needs the estimator and the matrix it actually sees, so explain.py splits the two, transforms the customers, and recovers the 35 transformed feature names. One detail matters for readability: the model is trained on standardised values, so a raw plot would say *Tenure* = -1.094. The scaler is inverted for **display only**, so the figures read *Tenure* = 1 while the Shapley values stay those of the model.

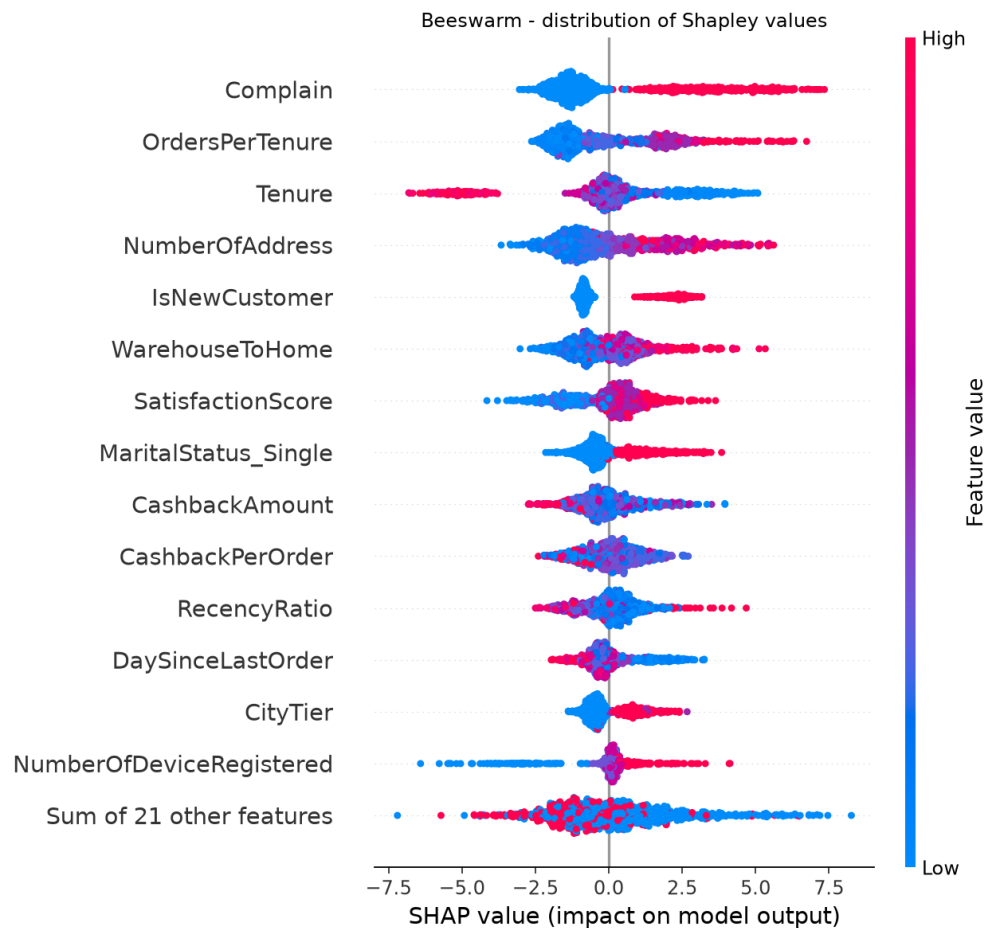
```
make explain          # explains the highest-risk customer
make explain CUSTOMER=54023  # explains one named customer
mlflow run . -e explain --env-manager local
```

### The property that makes SHAP trustworthy, checked in the test suite

Shapley values are **additive**: the base value plus every contribution must equal the model's raw output for that customer. `test_shapley_values_are_additive` asserts exactly that to 1e-6, so a broken explainer fails CI rather than producing a plausible-looking but wrong picture.

## 9.3 Global explanation - what drives churn overall





Figures 7-8 - Mean absolute Shapley value per feature, and the beeswarm showing the distribution and direction of every customer's contribution.

| Feature              | Mean  SHAP | Direction read from the beeswarm                                                        |
|----------------------|------------|-----------------------------------------------------------------------------------------|
| Complain             | 2.01       | A complaint (red) pushes strongly towards churn; no complaint protects                  |
| OrdersPerTenure      | 1.53       | High order rate relative to tenure raises risk - young, intense accounts churn          |
| Tenure               | 1.50       | The clearest signal: long tenure (red) pushes hard towards staying                      |
| NumberOfAddress      | 1.40       | More addresses raises risk                                                              |
| IsNewCustomer        | 1.19       | The engineered flag is the 5th strongest feature - the early-life churn seen in the EDA |
| WarehouseToHome      | 0.92       | Longer delivery distance raises risk                                                    |
| SatisfactionScore    | 0.91       | Low scores raise risk, with a wide spread                                               |
| MaritalStatus_Single | 0.75       | Single customers churn more                                                             |

This confirms the exploratory analysis and Part 1's expectations: **service friction** (complaints, delivery distance, satisfaction) and **relationship age** (tenure, the new-customer flag) dominate. Two of the top five features are engineered ones, which justifies the feature work of step 2.

## 9.4 Summary plot for each class

The model is binary, so it has one set of Shapley values: positive pushes towards *churn*, negative towards *no churn*. The class summaries are therefore mirror images, produced from the same values and their negation - the same information read from either side.

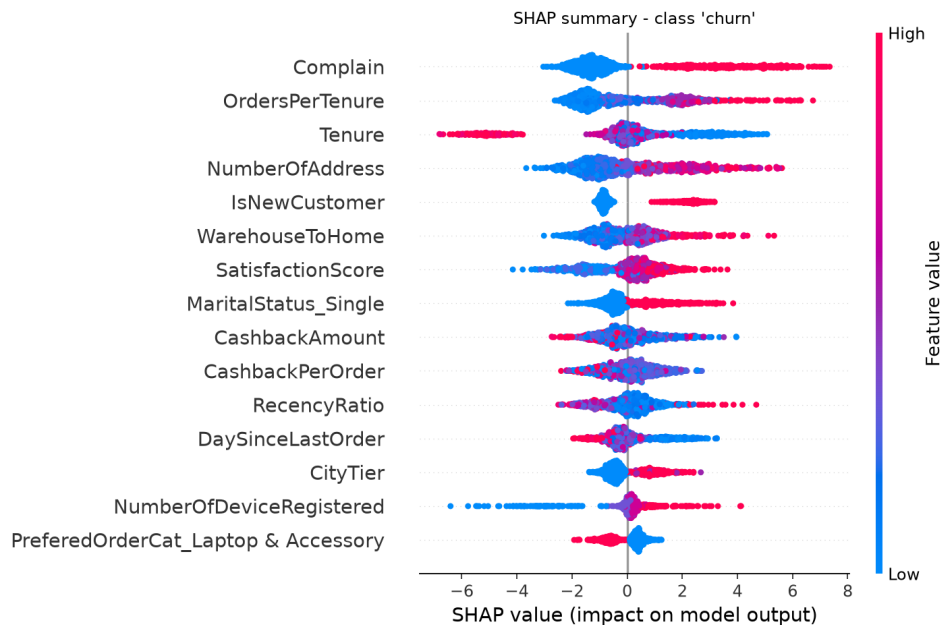
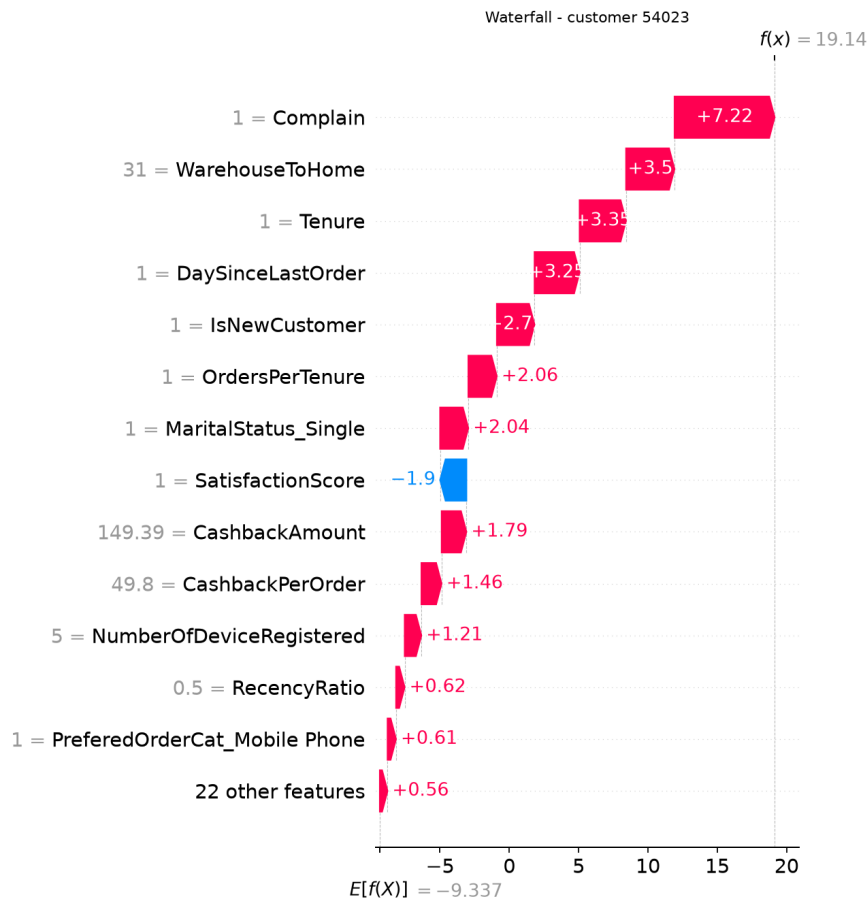


Figure 9 - Summary plot for the 'churn' class. The 'no churn' mirror is committed as `summary_class_no_churn.png`.

## 9.5 Explaining one customer

Customer **54023** is the highest-risk customer in the test split. The model's output for them is a raw score of 19.14 against a base value of -9.34 (log-odds), which is a churn probability of essentially 1. The waterfall shows how the 28.5 points were accumulated:



| Feature           | Value | SHAP  | Reading                                                      |
|-------------------|-------|-------|--------------------------------------------------------------|
| Complain          | 1     | +7.22 | They complained - by far the strongest single driver         |
| WarehouseToHome   | 31    | +3.50 | Far from the warehouse: slow deliveries                      |
| Tenure            | 1     | +3.35 | One month old, in the highest-risk window                    |
| DaySinceLastOrder | 1     | +3.25 | Recent order, yet still flagged - the other signals dominate |
| IsNewCustomer     | 1     | +2.72 | The early-life flag confirms the tenure effect               |
| SatisfactionScore | 1     | -1.90 | The only meaningful protective factor                        |

This is exactly the artefact Part 1 asked for: a retention manager can see *why* this customer is flagged - a complaint plus slow delivery on a one-month-old account - and choose a service gesture rather than a blind discount. The force plot below is the same explanation as a single line.

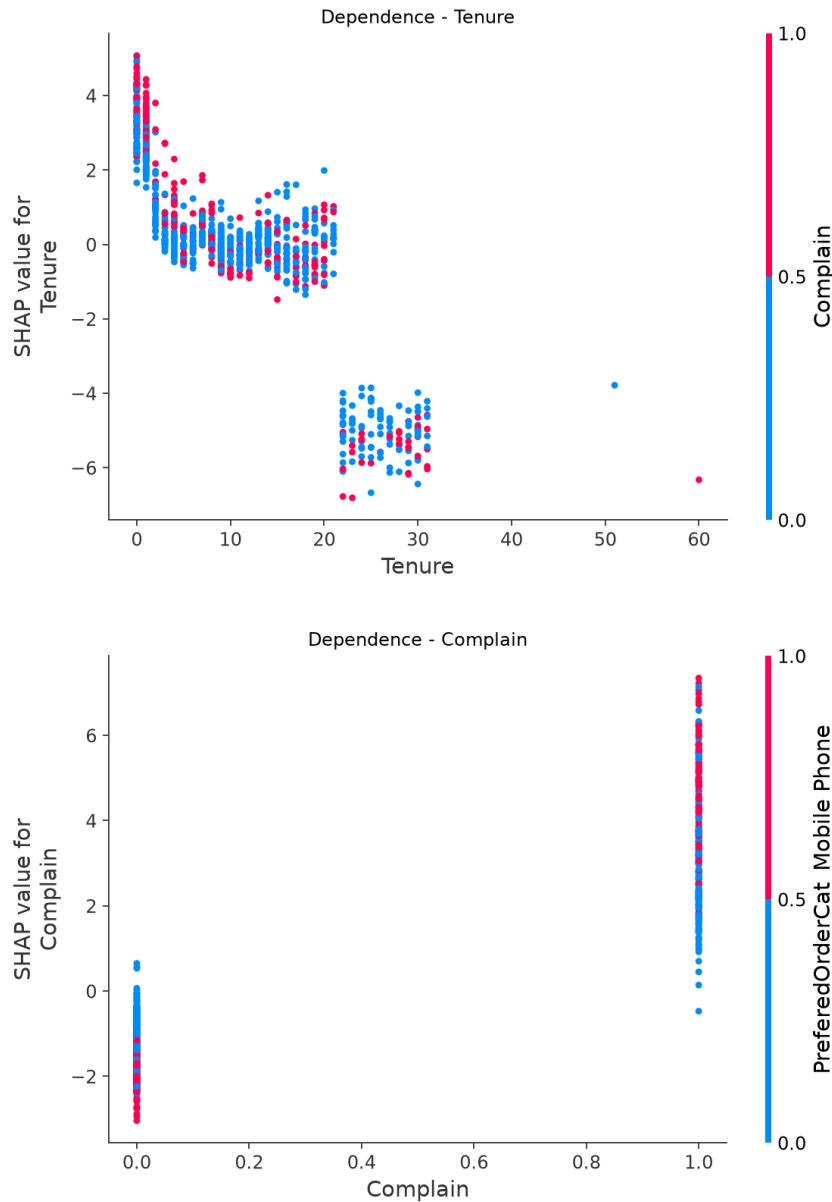


Figure 10 - Force plot for customer 54023: red forces push towards churn, blue towards staying.

## 9.6 All customers at once

Stacking every customer's force plot side by side gives the whole test population in one interactive figure, sorted by similarity. It is committed as **reports/figures/shap/force\_all\_customers.html** (400 customers; the plot is an HTML widget and cannot be embedded in a PDF). Opened in a browser it shows two clearly separated regimes - a small block of customers carried far above the base value by complaints and short tenure, and a long tail held below it by tenure and cashback.

## 9.7 Dependence plots



Figures 11-12 - Dependence plots for Tenure and Complain, coloured by the feature that interacts most with them.

| Observation                         | Numbers                                                                                       | Interpretation                                                                                                                           |
|-------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Tenure has a threshold, not a slope | Mean SHAP +3.05 for tenure $\leq 1$ month, -5.11 for tenure $\geq 22$ months                  | The model learned a step around 21 months rather than a linear effect - useful to marketing: the risk is concentrated in the first weeks |
| Complaints are close to binary      | Mean SHAP +3.58 when a complaint exists (342 of 1,126 customers) against -1.32 when none does | A complaint is worth roughly five points of log-odds; it is the single most actionable operational lever                                 |

| Observation | Numbers                               | Interpretation                                                                                                              |
|-------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Interaction | Colour of the tenure plot is Complain | Among short-tenure customers, those who also complained sit at the very top - the two risks compound rather than merely add |

## 9.8 From Shapley values to reason codes

`reason_codes()` turns the matrix into one readable line per customer, written to `reports/predictions_with_reasons.csv`. Drivers are ranked by the **size** of their contribution and carry their direction, because ranking by signed value alone would hand a safe customer three 'risk factors' that are merely their least protective features.

| CustomerID | churn_probability | risk_band | reason_1                            | reason_2                            |
|------------|-------------------|-----------|-------------------------------------|-------------------------------------|
| 52343      | 1.0               | HIGH      | Complain +5.78 (raises risk)        | OrdersPerTenure +5.60 (raises risk) |
| 54288      | 1.0               | HIGH      | Complain +5.77 (raises risk)        | Tenure +3.69 (raises risk)          |
| 54923      | 1.0               | HIGH      | NumberOfAddress +5.12 (raises risk) | Complain +4.20 (raises risk)        |

## 9.9 Going further - a fairness audit built on Shapley values

The course's ethics lab showed how explainability exposes a model leaning on a protected attribute. The same audit is applied here, since the dataset carries **Gender** and **MaritalStatus**. Two questions are asked: how much does the model *lean on* the attribute (mean absolute Shapley value of its encoded columns), and does the HIGH-risk flag fall unevenly on groups (selection rate, and its ratio to the most-flagged group - the disparate impact measure used in the lab).

| Attribute     | Group    | Customers | Mean  SHAP | Selection rate | Disparate impact | Actual churn |
|---------------|----------|-----------|------------|----------------|------------------|--------------|
| Gender        | Female   | 428       | 0.205      | 16.4%          | 0.95             | 16.1%        |
| Gender        | Male     | 698       | 0.205      | 17.2%          | 1.00             | 17.3%        |
| MaritalStatus | Divorced | 182       | 0.328      | 17.6%          | 0.66             | 17.0%        |
| MaritalStatus | Married  | 575       | 0.328      | 10.4%          | 0.39             | 10.3%        |
| MaritalStatus | Single   | 369       | 0.328      | 26.6%          | 1.00             | 27.1%        |

- **Gender is not driving the model.** Its mean |SHAP| of 0.205 is roughly a tenth of Complain's, and the selection rates differ by 0.8 of a point - a disparate impact of 0.95, comfortably above the 0.8 threshold used in the course lab.
- **Marital status looks unequal but is not disparate treatment.** Single customers are flagged 2.6 times as often as married ones, yet their actual churn rate is 27.1% against 10.3% - the selection rates track the real rates almost exactly, so the model is reflecting behaviour rather than penalising a group.
- **The distinction matters operationally, not just ethically.** A retention offer targeted by risk will reach single customers more often; the audit documents that this follows from behaviour, which is the evidence the governance bodies of Part 1 would ask for.
- **Limits.** Equal selection rates are only one fairness definition; a full audit would also test equalised odds per group and check proxies (a removed attribute can be reconstructed from others). The audit is committed as `reports/fairness_audit.csv` and reruns with `make explain`.

## 9.10 Limits of these explanations

- **Correlated features share credit.** Tenure, IsNewCustomer and OrdersPerTenure encode overlapping information, so their individual values are diluted; the group matters more than any single bar.
- **Explanation is not causation.** SHAP explains *the model*, not the world. It says the model raises risk when a complaint exists, not that removing complaints prevents churn - though the size of the effect makes that a hypothesis worth an experiment.
- **The model is near-perfect, so its explanations are unusually clean.** On messier real data the waterfall would be flatter and less quotable; section 10 explains why this dataset is optimistic.
- **Interventional vs. observational values.** TreeExplainer's default path-dependent values respect the tree structure rather than a causal graph; with a background dataset the interventional variant would answer a slightly different question.

## 10. Reading the results critically

A PR-AUC of 0.999 would be a red flag in production. Part 1 insisted on scepticism towards scores that look too good, so the same scepticism is applied here.

| Observation                            | Explanation                                                                                                                                                              | What it means                                                                                       |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Near-perfect separation                | The Kaggle dataset is a clean, static extract with strong, low-noise signals (tenure, complaints, cashback). Tree ensembles capture their interactions almost completely | Good for demonstrating the tooling; not evidence that churn is this predictable in reality          |
| No out-of-time validation              | The file has no date column, so the time-based split recommended in Part 1 is impossible; the stratified random split is the best available                              | Scores are optimistic. With real data the split would be by scoring date                            |
| Possible leakage cannot be ruled out   | The data dictionary does not state when columns such as DaySinceLastOrder or CashbackAmount were captured relative to the churn label                                    | In a real project these would go through the leakage checklist of Part 1 before any deployment      |
| Lift @10% stops at 5.93                | The top decile (112 customers) is smaller than the churning population (190), so a perfect ranking gives a lift of $1 / 0.169 = 5.93$                                    | Metric saturated; recall @10% (0.59) is capped the same way. A 20% cutoff would discriminate better |
| Logistic regression lags (PR-AUC 0.72) | The signal is in interactions and thresholds that a linear model cannot represent without manual feature engineering                                                     | Still the transparent baseline Part 1 asked for; the gap measures what the tree model adds          |

## 11. Testing and quality

21 pytest tests cover the whole pipeline. They run in CI and with make test:

| Area             | What is tested                                                                                                                                                       |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data preparation | category spellings merged; duplicate customers dropped; split sizes and stratification                                                                               |
| Features         | engineered columns finite, including tenure = 0; unseen categories and missing values handled by the fitted preprocessor                                             |
| Metrics          | lift and recall at k on a perfect ranking; metric set and values                                                                                                     |
| Training         | every candidate pipeline fits and returns probabilities; parameter overrides applied; a run logs parameters, metrics and plots; registration sets the champion alias |
| Inference        | risk band thresholds; one row per customer with a unique rank                                                                                                        |
| Plots            | all four evaluation figures are written                                                                                                                              |

Training tests use a throw-away MLflow store in a temporary directory, so running the suite never adds runs to the project's experiment. Line coverage is **76%**; what remains uncovered is mostly the command-line entry points, which make all exercises end to end.

## 12. Technical challenges encountered

The brief is about the technical challenges of AI projects. These are the ones this project actually hit, with their fixes - each is visible in the code.

| Problem                                                              | Cause                                                                                                                    | Fix                                                                                           |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Logging the model failed: 'untrusted types found: numpy.dtype'       | MLflow 3.16 serialises sklearn models with skops by default, and skops refuses some types present in a LightGBM pipeline | serialization_format="cloudpickle" in log_model, with a comment explaining why                |
| The REST endpoint returned 0/1 labels                                | The pyfunc wrapper calls predict() by default                                                                            | pyfunc_predict_fn="predict_proba"; the signature is inferred from the probabilities           |
| mlflow models serve exited immediately                               | MLflow 3 serves through uvicorn / FastAPI, which were not dependencies                                                   | uvicorn and fastapi added to the runtime dependencies                                         |
| MLflow warned that integer columns cannot hold missing values        | The signature was inferred with some numeric columns typed as integers                                                   | feature_frame() casts every numeric input to float64                                          |
| Committed MLflow outputs break after a clone                         | MLflow stores artefact locations as absolute paths                                                                       | scripts/relocate_mlflow.py rewrites them to the current checkout; run by make ui / make serve |
| Scripts wrote to different stores depending on the working directory | MLflow's default tracking location is relative to the current directory                                                  | TRACKING_URI and the artefact root are pinned in config.py                                    |
| pip tried to compile pyarrow from source                             | Caret pins (pyarrow ^17) excluded the newer versions that ship pre-built packages for Python 3.14                        | Version ranges with an upper bound on the major version only                                  |

| Problem                                           | Cause                                                                                               | Fix                                                                                                          |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| SHAP plots showed z-scores instead of real values | The model is trained on standardised features, so the explanations displayed<br>Tenure = -1.094     | <code>display_frame()</code> inverts the scaler for display only; the Shapley values stay those of the model |
| Shapley output shape differs between versions     | LightGBM binary classifiers return either one matrix or one per class depending on the SHAP version | <code>shapley_values()</code> normalises both to the positive class before any plot is drawn                 |
| The MLflow UI exhausted memory on a small machine | mlflow ui starts four worker processes by default (about 230 MB each)                               | make ui passes <code>--workers 1</code>                                                                      |

## 13. Conclusion

The project delivers every item of the brief: versioned code on GitHub with CI, a cookiecutter structure with one script per pipeline step, PEP8 tooling passing cleanly, Poetry-managed dependencies, generated Sphinx documentation, and MLflow used end to end - ten tracked runs, a packaged MLflow project, a registered champion model and a working local REST endpoint.

Part 3 completes the picture. The champion no longer answers only *who* is about to churn but *why*: 11 SHAP artefacts, reason codes for all 1,126 customers, and a fairness audit showing that gender is not driving the score. The explanation layer is tested for additivity and rebuilt by a single make target, so it stays true as the model is retrained.

It also closes the loop with Part 1. The choices framed there - rank-based metrics for an imbalanced target, one transformation path shared by training and serving, a registry that makes promotion and rollback a single step, probabilities and risk bands instead of bare labels, and scepticism towards scores that look too good - are now implemented in code, not only described.

## 14. References

- Verma, A. **E-Commerce Customer Churn Analysis and Prediction** (dataset). Kaggle. [kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction](https://kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction)
- DrivenData. **Cookiecutter Data Science**. [drivendata.github.io/cookiecutter-data-science/](https://drivendata.github.io/cookiecutter-data-science/)
- van Rossum, G., Warsaw, B., Coghlan, N. **PEP 8 - Style Guide for Python Code**. [peps.python.org/pep-0008/](https://peps.python.org/pep-0008/)
- MLflow documentation - **Tracking, Projects, Model Registry, Deployment**. [mlflow.org/docs/latest/ml/](https://mlflow.org/docs/latest/ml/)
- Sphinx documentation. [sphinx-doc.org/en/master/](https://sphinx-doc.org/en/master/) ; napoleon extension for Google-style docstrings.
- Poetry documentation. [python-poetry.org/docs/](https://python-poetry.org/docs/)
- black, flake8 and pylint documentation ([pypi.org/project/black/](https://pypi.org/project/black/), [flake8](https://pypi.org/project/flake8/), [pylint](https://pypi.org/project/pylint/)).
- Pedregosa, F. et al. (2011). **Scikit-learn: Machine Learning in Python**. JMLR 12.
- Ke, G. et al. (2017). **LightGBM: A Highly Efficient Gradient Boosting Decision Tree**. NeurIPS.
- Lundberg, S. and Lee, S.-I. (2017). **A Unified Approach to Interpreting Model Predictions**. NeurIPS - the SHAP framework.
- Lundberg, S. et al. (2020). **From local explanations to global understanding with explainable AI for trees**. Nature Machine Intelligence - the exact TreeExplainer algorithm used here.
- SHAP documentation and API reference. [shap.readthedocs.io](https://shap.readthedocs.io)

- Shapley, L. S. (1953). **A value for n-person games**. Contributions to the Theory of Games II.
- Barocas, S., Hardt, M., Narayanan, A. (2023). **Fairness and Machine Learning**. [fairmlbook.org](https://fairmlbook.org) - selection rate, disparate impact and their limits.
- Course material, **AI Ethical Challenges: Interpretability & Explainability lab** and **Bias & Fairness lab**, EPITA 2026 - the SHAP audit approach applied in section 9.9.
- Saito, T. and Rehmsmeier, M. (2015). **The precision-recall plot is more informative than the ROC plot on imbalanced datasets**. PLoS ONE.
- GitHub Actions documentation. [docs.github.com/actions](https://docs.github.com/actions)
- Group report, **Graded Project - Part 1: functional framing of the RetailGenius churn project**, 2026.

### **Use of generative AI**

Generative AI (Claude, Anthropic) was used as an assistant to help write and debug code, structure this report and review the project against the brief. All design choices, results and text were reviewed and validated by the group.